

SOLUCIONANDO SUDOKU COM UM ALGORÍTMO DE BACKTRACKING

Lucas Ângelo Câmara da Silva
Matrícula: 202410367011

Gusthavo Cassimiro Lemos Pereira
Matrícula: 202410330211

Instituto Politécnico, Universidade do Estado do Rio de Janeiro
Disciplina: Projeto e Análise de Algoritmos
Professor: Prof^ª. Dr^ª. Camila Martins Saporetti

Julho, 2025

INTRODUÇÃO

É inegável o papel fundamental que a computação desempenha em nossas vidas hoje. De tarefas complexas de pesquisa científica a atividades do dia a dia, os computadores transformam a maneira como interagimos com o mundo. Essa vasta aplicação inclui, claro, o entretenimento.

Um exemplo popular de como a computação se entrelaça com o entretenimento é o jogo de Sudoku. Para quem não conhece, o Sudoku é um passatempo mundialmente famoso. O objetivo é preencher uma grade 9×9 com números de 1 a 9, sem repetir dígitos em nenhuma linha, coluna ou nas nove subgrades 3×3 .

Embora as regras sejam simples, o Sudoku pode variar de 'fácil' a 'muito desafiador', com os quebra-cabeças mais difíceis geralmente tendo mais células vazias. Para muitos, a complexidade pode ser um obstáculo. É aqui que a computação entra. Este seminário irá explorar como algoritmos podem ser usados para resolver esses quebra-cabeças, transformando um desafio manual em um problema computacionalmente solúvel.

METODOLOGIA

O foco principal da nossa discussão será o algoritmo de Backtracking. O Backtracking é uma técnica genérica e um dos métodos mais eficientes para resolver problemas de otimização combinatória, como o Sudoku. O algoritmo começa construindo uma solução parcial. Se essa solução se torna inviável ou viola uma restrição, o algoritmo 'retrocede' – daí o nome 'backtracking' – para a última decisão tomada e explora uma alternativa. Isso permite descartar grandes

blocos de soluções potenciais de uma vez, tornando-o bastante eficiente.

Para aplicar o backtracking ao Sudoku, precisamos representar o tabuleiro de forma computacional. Um tabuleiro de Sudoku é uma matriz 9×9 de inteiros. Cada célula vazia é considerada uma variável que precisa ser preenchida. O algoritmo mantém para cada célula vazia uma lista de dígitos compatíveis, identificados escaneando a linha, coluna e subgrade da célula.

Vamos descrever o processo passo a passo:

- **Inicialização:** Para cada célula vazia (i, j) , criamos uma lista L_{ij} de dígitos compatíveis (de 1 a 9). As células são então ordenadas com base na cardinalidade (tamanho) dessas listas, do menor para o maior. Células com menos opções são preenchidas primeiro.
- **Preenchimento Tentativo:** O algoritmo tenta preencher a célula vazia com a menor lista de opções, escolhendo o primeiro dígito disponível dessa lista. Isso forma uma 'solução parcial'.
- **Verificação de Validade:** Após cada preenchimento, o algoritmo verifica se a solução parcial ainda é válida, ou seja, se não há violações das regras do Sudoku (números repetidos na linha, coluna ou subgrade).
- **Backtracking:** Se uma violação é encontrada, o algoritmo retrocede para a última célula preenchida. Ele descarta a escolha atual e tenta o próximo dígito disponível para essa célula. Se não houver mais dígitos disponíveis para essa célula, ele retrocede mais uma vez, para a célula anterior.

- **Sucesso ou Falha:** Esse processo recursivo continua até que todas as células estejam preenchidas sem violações (solução encontrada) ou até que o algoritmo tenha retrocedido até o ponto inicial e esgotado todas as possibilidades, indicando que nenhuma solução única existe.

A complexidade de tempo do algoritmo de backtracking para o Sudoku é de $O(n^3)$, onde n representa o tamanho do lado da subgrade (no Sudoku 9×9 , $n = 3$ para as subgrades 3×3), mas pode ser mais precisamente expressa como $O(9^3)$ para um tabuleiro 9×9 .

APROFUNDAMENTO

Embora o backtracking seja eficaz, ele não é a única forma de atacar o problema do Sudoku. Outras técnicas genéricas de otimização combinatória incluem:

- **Simulated Annealing (Recozimento Simulado):** Inspirado em modelos da física estatística, este método explora o espaço de soluções de forma mais ampla, aceitando até mesmo movimentos 'ruins' no início para evitar mínimos locais, e gradualmente 'resfria' para convergir em uma solução. Em testes, ele obteve sucesso em 100% dos puzzles fáceis e médios, mas apenas 3% nos 'Top 95' mais difíceis.
- **Alternating Projections (Projeções Alternadas):** Este método, enraizado na programação convexa, busca um ponto na interseção de um conjunto de condições, transformando o problema discreto em um problema de otimização contínua. Sua performance nos puzzles 'Top 95' foi de 41% de sucesso, superando o simulated annealing nessas instâncias mais desafiadoras.

A beleza do backtracking reside em sua aplicabilidade a uma vasta gama de problemas combinatórios. Por exemplo, ele é amplamente utilizado para resolver jogos de palavras cruzadas. A lógica é similar: preencher células com letras, verificando se as palavras formadas correspondem às pistas e se as interseções estão corretas. Se uma letra ou palavra não se encaixa, o algoritmo retrocede e tenta uma alternativa.

A inteligência artificial tem permitido que computadores resolvam diversos jogos. Além de Sudoku e palavras cruzadas, algoritmos de busca, incluindo variações do backtracking, são usados em jogos como o Xadrez e o jogo da Velha. No xadrez, por exemplo, árvores de busca são exploradas para prever movimentos e contramovimentos, embora a complexidade computacional do xadrez seja significativamente maior que a do Sudoku devido ao vasto espaço de estados.

CONCLUSÃO

Em nossos estudos, o algoritmo de backtracking se mostrou o mais eficaz para resolver quebra-cabeças de Sudoku. Ele consegue resolver infalivelmente um puzzle de Sudoku ou determinar que uma solução única não existe. Isso ocorre porque, apesar das aparências, os puzzles de Sudoku 9×9 são computacionalmente 'relativamente benignos', permitindo que o backtracking encontre a solução sem a complexidade excessiva que enfrentaria em problemas combinatórios de dimensões muito mais altas. Sua capacidade de 'podar' grandes subárvores de soluções inviáveis o torna superior a métodos como simulated annealing, especialmente em puzzles mais difíceis.

Para recapitular, vimos como a computação, em particular o algoritmo de Backtracking, oferece uma solução poderosa e eficiente para o desafiador jogo de Sudoku. Exploramos a mecânica do backtracking, como ele representa o Sudoku e como ele sistematicamente busca e valida soluções. Também destacamos sua versatilidade, mostrando sua aplicação em palavras cruzadas e contrastando-o com outras técnicas como Simulated Annealing e Alternating Projections.

REFERÊNCIAS

- ERIC C. CHI; KENNETH LANGE. **Techniques for Solving Sudoku Puzzles**. Ithaca, Nova York, Estados Unidos: Cornell University, 2012. Disponível em: <<https://arxiv.org/abs/1203.2295>>. Acesso em: jun. 2025.
- HERIMANTO; SITORUS, P.; ZAMZAMI, E. M. **An Implementation of Backtracking Algorithm for Solving A Sudoku-Puzzle Based on Android**. Bristol, Reino Unido: IOP Publishing, 2020. Disponível em: <<https://iopscience.iop.org/article/10.1088/1742-6596/1566/1/012038/meta>>. Acesso em: jun. 2025.
- NURDIN et al. **The Implementation of Backtracking Algorithm on Crossword Puzzle Games Based on Android**. Bristol, Reino Unido: IOP Publishing, 2019. Disponível em: <<https://iopscience.iop.org/article/10.1088/1742-6596/1363/1/012075/meta>>. Acesso em: jun. 2025.